

# ProjectHub API, JWT Workflow, and Spring Boot Annotation Notes

Written in simple notes format from your backend and frontend folders only. No table/box layout.

## Files I referred from your project

- backend/demo/demo/src/main/java/com/example/demo/controller/\*.java
- backend/demo/demo/src/main/java/com/example/demo/security/JWTUtil.java
- backend/demo/demo/src/main/java/com/example/demo/security/JWTFilter.java
- backend/demo/demo/src/main/java/com/example/demo/config/SecurityConfig.java
- backend/demo/demo/src/main/java/com/example/demo/service/UserService.java
- backend/demo/demo/src/main/java/com/example/demo/service/ProjectService.java
- frontend/projecthub/front/src/Login.jsx, ResetPassword.jsx
- frontend/projecthub/front/src student pages: Dashboard, SubmitProject, Certificates, Profile, Notifications
- frontend/projecthub/front/src faculty pages: FacultyDashboard, AssignedStudents, ReviewProjects, VerifyCertificates, FacultyNotifications
- frontend/projecthub/front/src admin pages: AdminDashboard, ManageUsers, AdminSettings, ViewSubmissions, AdminNotifications

## Spring Boot annotations used

- @SpringBootApplication - Used in DemoApplication.java. It starts the Spring Boot app and enables auto configuration and component scanning.
- @RestController - Used in controller files. It makes classes work as REST API controllers and returns JSON/text responses.
- @RequestMapping - Used for base paths like /users, /projects, /certificates, /admin, /deadline-rules, /landing.
- @GetMapping - Used for reading data: dashboard, profile, project list, certificate list, notifications, settings.
- @PostMapping - Used for creating/action APIs: login, register, upload file, create project, create certificate, forgot password.
- @PutMapping - Used for updates: profile save, edit project/certificate, review project, verify certificate, assign faculty.
- @DeleteMapping - Used for delete actions: user, project, certificate, notification, deadline rule.
- @RequestBody - Reads JSON body from frontend into DTO/entity/map.
- @PathVariable - Reads values from URL path, for example {id}, {studentId}, {facultyId}, {fileName}.
- @RequestParam - Reads query parameters and multipart file fields, for example status, feedback, remarks, subject, file.
- @CrossOrigin("") - Allows React frontend to call Spring Boot APIs.
- @Service - Used in UserService and ProjectService for business logic.
- @Autowired - Injects service, repository, JWT, password encoder, and mail dependencies.
- @Configuration and @Bean - Used for SecurityConfig, PasswordConfig, and OpenApiConfig.
- @Component - Used for JWTUtil and JWTFilter so Spring can manage them.
- @Entity, @Table, @Id, @GeneratedValue - Used for MySQL/JPA models like User, Student, Faculty, Admin, Project.
- @Document and @Id - Used for MongoDB models like ProjectDocument, CertificateDocument, NotificationDocument, DeadlineRule.
- @OneToOne, @ManyToOne, @JoinColumn - Used for relations between User, Student, Faculty, Admin, Project, AuditLog.
- @Valid, @NotBlank, @NotNull, @Size - Used for validating project request/document fields.
- @Transactional - Used in UserService for safe multi-step database work.

- `@RestControllerAdvice` and `@ExceptionHandler` - Used in `GlobalExceptionHandler` for common error response handling.
- `@Query` and `@Param` - Used in `UserRepository` for custom normalized email lookup.
- `@Value` - Used in `UserService` to read `app.frontend-url` and mail username from properties.

## JWT workflow

- 1 In Login.jsx, the user enters email and password. Password is encrypted in frontend using AES-GCM before sending.
- 2 Login.jsx sends POST /users/login with email and encrypted password.
- 3 UserController.login receives AuthRequest and passes it to UserService.login.
- 4 UserService normalizes the email, finds the user, decrypts enc:v2 password if needed, and checks password using BCryptPasswordEncoder.
- 5 JWTUtil.generateToken creates a JWT using HS256. It stores email as subject and role as a claim. Token expiry is one hour.
- 6 Login.jsx stores token, userId, role, and fullName in localStorage.
- 7 After login, frontend navigates by role: student goes to /dashboard, faculty to /faculty-dashboard, admin to /admin-dashboard.
- 8 For protected pages, frontend sends Authorization: Bearer token in fetch headers.
- 9 JWTFilter skips public APIs like login, forgot password, reset password, register, landing click, OPTIONS, Swagger, error, and project download.
- 10 For protected APIs, JWTFilter validates token, extracts role, creates ROLE\_STUDENT / ROLE\_FACULTY / ROLE\_ADMIN, and sets SecurityContext.
- 11 SecurityConfig decides which API needs which role.

## Common auth and button APIs

- Sign In button or Enter key - Login.jsx handleLogin calls POST /users/login. Auth: Public. Login API returns JWT token, user id, role, and full name.
- Forgot password? button - Login.jsx trackAuthAction calls GET /landing/click?action=forgot\_password\_click. Auth: Public. This is a backend-visible tracking API for the button click.
- Back to Sign In button in forgot mode - Login.jsx trackAuthAction calls GET /landing/click?action=back\_to\_signin\_click. Auth: Public. Tracks the back button click.
- Send Reset Link button - Login.jsx handleForgotPassword calls POST /users/forgot-password. Auth: Public. Creates password reset token and sends/returns reset link.
- Open Reset Link button - Login.jsx trackAuthAction calls GET /landing/click?action=reset\_password\_link\_click. Auth: Public. Tracks reset link button before opening reset page.
- Change Password button or Enter key - ResetPassword.jsx handleChangePassword calls POST /users/reset-password. Auth: Public. Validates token and changes student password.
- Back to Sign In button in reset page - ResetPassword.jsx trackAuthAction calls GET /landing/click?action=reset\_back\_to\_signin\_click. Auth: Public. Tracks reset page back button.

## Module 1: Student APIs

- Dashboard page load - Dashboard.jsx calls GET /projects/analytics/student/{userId}. Auth: Bearer token. Loads student analytics cards.
- Dashboard page load - Dashboard.jsx calls GET /projects/mongo/student/{userId}. Auth: Bearer token. Loads student projects.
- Dashboard page load - Dashboard.jsx calls GET /certificates/student/{userId}. Auth: Bearer token. Loads student certificates.
- Dashboard page load - Dashboard.jsx calls GET /users/student/profile/{userId}. Auth: Bearer token. Loads profile summary.
- Export PDF button - Dashboard.jsx handleExportPDF calls GET dashboard APIs again. Auth: Bearer token. Refreshes data before frontend PDF export.
- Submit Project page load - SubmitProject.jsx calls GET /projects/mongo/student/{userId}. Auth: Bearer token. Shows submission history.
- Submit Project page load - SubmitProject.jsx calls GET /users/filter/role?role=FACULTY. Auth: Bearer token. Loads faculty dropdown.
- Submit Project page load - SubmitProject.jsx calls GET /deadline-rules/type/Project. Auth: Bearer token. Loads project deadline rules.
- Submit Project button - SubmitProject.jsx handleSubmit calls POST /projects/upload. Auth: Bearer token. Uploads selected project file.
- Submit Project button - SubmitProject.jsx handleSubmit calls POST /projects/mongo. Auth: Bearer token. Creates project record.
- Edit Project save - SubmitProject.jsx handleSubmit calls PUT /projects/mongo/update/{id}. Auth: Bearer token. Updates existing project.
- Delete Project icon - SubmitProject.jsx handleDeleteProject calls DELETE /projects/mongo/{id}. Auth: Bearer token. Deletes project.
- View/download project file - SubmitProject.jsx handleDownloadProjectFile calls GET /projects/download/{fileName}. Auth: Public file route. Downloads/views uploaded file.
- Certificates page load - Certificates.jsx calls GET /certificates/student/{userId}. Auth: Bearer token. Loads student certificates.
- Certificates page load - Certificates.jsx calls GET /users/filter/role?role=FACULTY. Auth: Bearer token. Loads faculty dropdown.
- Certificates page load - Certificates.jsx calls GET /deadline-rules/type/Certificate. Auth: Bearer token. Loads certificate deadline rules.
- Submit Certificate button - Certificates.jsx handleCertificateSubmit calls POST /projects/upload. Auth: Bearer token. Uploads certificate file.
- Submit Certificate button - Certificates.jsx handleCertificateSubmit calls POST /certificates. Auth: Bearer token. Creates certificate record.
- Edit Certificate save - Certificates.jsx handleCertificateSubmit calls PUT /certificates/{id}. Auth: Bearer token. Updates certificate.
- Delete Certificate icon - Certificates.jsx handleDelete calls DELETE /certificates/{id}. Auth: Bearer token. Deletes certificate.
- View certificate file - Certificates.jsx handleViewCertificate calls GET latestCert.fileURL. Auth: Public file route. Fetches file blob.
- Save Profile button - Profile.jsx handleSave calls PUT /users/student/profile/{userId}. Auth: Bearer token. Updates student profile.
- Notifications page - Notifications.jsx calls GET /users/notifications/{userId}. Auth: Bearer token. Loads notifications.
- Mark all read button - Notifications.jsx handleMarkRead calls PUT /users/notifications/read/{id}. Auth: Bearer token. Marks notifications read.
- Delete notification icon - Notifications.jsx handleDelete calls DELETE /users/notifications/{id}. Auth: Bearer token. Deletes notification.

## Module 2: Faculty APIs

- Faculty dashboard load - FacultyDashboard.jsx calls GET /projects/mongo/faculty/{userId}. Auth: Bearer token. Loads assigned projects.
- Faculty dashboard load - FacultyDashboard.jsx calls GET /certificates/faculty/{userId}. Auth: Bearer token. Loads assigned certificates.
- Assigned Students page load - AssignedStudents.jsx calls GET /users/faculty/{facultyUserId}/students. Auth: Bearer token. Loads assigned students.
- Assigned Students page load - AssignedStudents.jsx calls GET /projects/mongo/faculty/{userId}. Auth: Bearer token. Loads projects for assigned students.
- Review Projects page/filter click - ReviewProjects.jsx fetchProjects calls GET /projects/mongo/faculty/{userId}. Auth: Bearer token. Loads projects for review.
- Review Projects page/filter click - ReviewProjects.jsx fetchProjects calls GET /deadline-rules/type/Project. Auth: Bearer token. Loads project rules.
- Approve/Reject Project buttons - ReviewProjects.jsx handleReview calls PUT /projects/mongo/review/{id}?status=&feedback=&grade=. Auth: Bearer token. Submits project review.
- Download Project file button - ReviewProjects.jsx handleDownloadProjectFile calls GET /projects/download/{fileName}. Auth: Public file route. Downloads project file.
- Verify Certificates page/filter click - VerifyCertificates.jsx fetchCertificates calls GET /certificates/faculty/{userId}. Auth: Bearer token. Loads certificates to verify.
- Verify Certificates page/filter click - VerifyCertificates.jsx fetchCertificates calls GET /deadline-rules/type/Certificate. Auth: Bearer token. Loads certificate rules.
- Approve/Reject Certificate buttons - VerifyCertificates.jsx handleCertificateReview calls PUT /certificates/{id}/verify?status=&remarks=. Auth: Bearer token. Submits certificate verification.
- Download certificate file button - VerifyCertificates.jsx handleDownload calls GET fileURL. Auth: Public file route. Downloads certificate file.
- Save Profile button - Profile.jsx handleSave role=FACULTY calls PUT /users/faculty/profile/{userId}. Auth: Bearer token. Updates faculty profile.
- Notifications page - FacultyNotifications.jsx calls GET /users/notifications/{userId}. Auth: Bearer token. Loads notifications.
- Mark all read button - FacultyNotifications.jsx handleMarkAllRead calls PUT /users/notifications/read/{id}. Auth: Bearer token. Marks notifications read.
- Delete notification button - FacultyNotifications.jsx handleDelete calls DELETE /users/notifications/{id}. Auth: Bearer token. Deletes notification.

## Module 3: Admin APIs

- Admin dashboard load/refresh - AdminDashboard.jsx calls GET /users/admin/analytics. Auth: Bearer token. Loads analytics.
- Admin dashboard load/refresh - AdminDashboard.jsx calls GET /users/audit-logs. Auth: Bearer token. Loads audit logs.
- Manage Users page load/refresh - ManageUsers.jsx fetchUsers calls GET /users. Auth: Bearer token. Loads all users.
- Edit icon - ManageUsers.jsx inline onClick calls GET /users/student/profile/{user.id}. Auth: Bearer token. Loads student profile for edit modal.
- Edit icon - ManageUsers.jsx inline onClick calls GET /users/faculty/profile/{user.id}. Auth: Bearer token. Loads faculty profile for edit modal.
- Delete user icon - ManageUsers.jsx handleDelete calls DELETE /users/{id}. Auth: Bearer token. Deletes user.
- Save Changes button - ManageUsers.jsx handleSave calls PUT /users/{id}. Auth: Bearer token. Updates user/profile.
- Add User Save Changes button - ManageUsers.jsx handleCreateUser calls POST /auth/register. Auth: Public in config. Creates new user.
- Add User Save Changes button - ManageUsers.jsx handleCreateUser calls PUT /users/{createdUser.id}. Auth: Bearer token. Saves extra role profile fields.
- Bulk Upload button - ManageUsers.jsx handleBulkUpload calls POST /users/bulk-upload. Auth: Bearer token. Bulk registers users from Excel.
- View Submissions page/filter click - ViewSubmissions.jsx fetchData calls GET /admin/projects. Auth: Bearer token. Loads all projects.
- View Submissions page/filter click - ViewSubmissions.jsx fetchData calls GET /admin/certificates. Auth: Bearer token. Loads all certificates.
- Settings page load/refresh - AdminSettings.jsx calls GET /deadline-rules. Auth: Bearer token. Loads deadline rules.
- Settings page load/refresh - AdminSettings.jsx calls GET /users/audit-logs. Auth: Bearer token. Loads audit logs.
- Settings page load/refresh - AdminSettings.jsx calls GET /users/settings. Auth: Bearer token. Loads settings.
- Settings page load/refresh - AdminSettings.jsx calls GET /users/filter/role?role=FACULTY. Auth: Bearer token. Loads faculty dropdown.
- Save Settings button - AdminSettings.jsx saveSettings calls POST /users/settings. Auth: Bearer token. Saves settings.
- Save Deadline button - AdminSettings.jsx saveDeadlineRule calls POST /deadline-rules. Auth: Bearer token. Creates/updates deadline.
- Assign Faculty button - AdminSettings.jsx handleAssignFaculty calls PUT /users/assign-faculty/{id}?subject=. Auth: Bearer token. Assigns subject to faculty.
- Assign Students button - AdminSettings.jsx handleAssignStudentsByDepartments calls PUT /users/admin/assign-students-by-department?department=&section=&facultyUserId=. Auth: Bearer token. Bulk assigns students.
- Admin notifications page - AdminNotifications.jsx calls GET /users/notifications/{userId}. Auth: Bearer token. Loads notifications.
- Mark all read button - AdminNotifications.jsx handleMarkAllRead calls PUT /users/notifications/read/{id}. Auth: Bearer token. Marks notifications read.
- Delete notification - AdminNotifications.jsx handleDelete calls DELETE /users/notifications/{id}. Auth: Bearer token. Deletes notification.

## Small note

Some buttons in the frontend are UI-only. For example sidebar navigation, close modal, open dropdown, show/hide password, pagination, tab switching, and date picker clicks do not call backend APIs unless the handler also contains fetch(). I listed the buttons/screens that actually call backend APIs or tracking APIs in your frontend code.